

295. Find Median from Data Stream **Hard**

Acceptance: 54.1% | [LeetCode](#)

The **median** is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for `arr = [2,3,4]`, the median is `3`.
- For example, for `arr = [2,3]`, the median is $(2 + 3) / 2 = 2.5$.

Implement the MedianFinder class:

- `MedianFinder()` initializes the `MedianFinder` object.
- `void addNum(int num)` adds the integer `num` from the data stream to the data structure.
- `double findMedian()` returns the median of all elements so far. Answers within `10-5` of the actual answer will be accepted.

Example 1:

```
**Input**
["MedianFinder", "addNum", "addNum", "findMedian",
 "addNum", "findMedian"]
[[], [1], [2], [], [3], []]
**Output**
[null, null, null, 1.5, null, 2.0]
```

****Explanation****

```
MedianFinder medianFinder = new MedianFinder();
medianFinder.addNum(1);    // arr = [1]
medianFinder.addNum(2);    // arr = [1, 2]
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
medianFinder.addNum(3);    // arr[1, 2, 3]
medianFinder.findMedian(); // return 2.0
```

Constraints:

- $-105 \leq \text{num} \leq 105$
- There will be at least one element in the data structure before calling `findMedian`.
- At most $5 * 10^4$ calls will be made to `addNum` and `findMedian`.

Follow up:

- If all integer numbers from the stream are in the range `[0, 100]`, how would you optimize your solution?
- If 99% of all integer numbers from the stream are in the range `[0, 100]`, how would you optimize your solution?

Claude Opus Solution

Find Median from Data Stream

1. Intuition

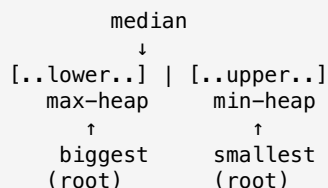
The **key insight**: To find a median efficiently, we don't need the entire list sorted—we only need access to the **middle element(s)**.

We can split the data into two halves: - A **lower half** (all the smaller numbers) - An **upper half** (all the larger numbers)

If we can always look at the **largest element of the lower half** and the **smallest element of the upper half**, we have everything needed to compute the median.

This is exactly what **two heaps** give us: - A **max-heap** for the lower half → root is the largest of the small numbers - A **min-heap** for the upper half → root is the smallest of the large numbers

By keeping the heaps **balanced** (sizes differ by at most 1), the median is either: - The root of the larger heap (odd total count), or - The average of both roots (even total count)



2. Approach (Pseudocode)

INIT:

```
max_heap = [] # lower half (store negatives to simulate max-heap)
min_heap = [] # upper half
```

`addNum(num)`:

```
# Goal: keep max_heap.size == min_heap.size OR max_heap.size == min_heap.size + 1
```

```

# Step 1: Always push into max_heap, but route the largest element to min_heap
# (heappushpop pushes num, then pops the max → goes to min_heap)
push -num into max_heap
move max_heap's max → min_heap

# Step 2: Rebalance if min_heap got bigger
if min_heap.size > max_heap.size:
    move min_heap's min → max_heap

findMedian():
    if max_heap.size > min_heap.size:
        return max_heap.top          # odd count: extra in max_heap
    else:
        return (max_heap.top + min_heap.top) / 2  # even count

```

3. Python Solution

```

from heapq import heappush, heappop

class MedianFinder:
    def __init__(self):
        # Python's heapq is a MIN-heap by default.
        # To simulate a MAX-heap, we store NEGATED values.
        #
        # max_heap: lower half of numbers (we store negatives)
        # → -max_heap[0] gives the LARGEST number in the lower half
        # min_heap: upper half of numbers (stored normally)
        # → min_heap[0] gives the SMALLEST number in the upper half
        self.max_heap = [] # lower half (negated)
        self.min_heap = [] # upper half

    def addNum(self, num: int) -> None:
        # --- Step 1: Insert into lower half (max_heap) ---
        # Push num into max_heap (negated), then pop the current max.
        # The popped max is the "overflow" that belongs in the upper half.
        heappush(self.max_heap, -num)          # add to lower half
        overflow = -heappop(self.max_heap)      # extract current largest

        # Push that overflow into the upper half (min_heap).
        heappush(self.min_heap, overflow)

        # --- Step 2: Rebalance ---
        # After step 1, min_heap may have one more element than max_heap.
        # We allow max_heap to be the "bigger" one (holds the extra element
        # when total count is odd). So if min_heap grows too large, move
        # its smallest element back into max_heap.
        if len(self.min_heap) > len(self.max_heap):
            smallest_upper = heappop(self.min_heap) # smallest of upper half
            heappush(self.max_heap, -smallest_upper) # store negated

    def findMedian(self) -> float:
        # Two invariants are maintained:
        # max_heap.size == min_heap.size      (even total) OR
        # max_heap.size == min_heap.size + 1  (odd total, extra in max_heap)

        if len(self.max_heap) > len(self.min_heap):
            # Odd count → the single middle element is the root of max_heap.
            return -self.max_heap[0]
        else:
            # Even count → average the two middle elements.
            # -max_heap[0] = largest of lower half
            # min_heap[0] = smallest of upper half
            return (-self.max_heap[0] + self.min_heap[0]) / 2

```

Note on the reference solution: It uses `heappushpop` (a single atomic operation) instead of separate `heappush` / `heappop`, which is slightly more efficient. My version above splits them out for clarity. The reference also conditionally chooses which heap to push into first—both achieve the same balanced invariant.

Cleaner version matching the reference style:

```
from heapq import heappush, heappushpop

class MedianFinder:
    def __init__(self):
        self.min_heap, self.max_heap = [], [] # upper half, lower half (negated)

    def addNum(self, num: int) -> None:
        # If heaps are equal size, the new element should grow max_heap.
        # heappushpop(min_heap, num): push num, pop smallest -> that smallest
        # belongs in the lower half, so push it (negated) into max_heap.
        if len(self.min_heap) < len(self.max_heap):
            heappush(self.min_heap, -heappushpop(self.max_heap, -num))
        else:
            heappush(self.max_heap, -heappushpop(self.min_heap, num))

    def findMedian(self) -> float:
        if len(self.min_heap) == len(self.max_heap):
            return (self.min_heap[0] - self.max_heap[0]) / 2
        return -self.max_heap[0] # max_heap is always the larger one here
```

4. Complexity Analysis

Operation	Time	Reason
addNum	$O(\log n)$	Heap push/pop operations are logarithmic in heap size
findMedian	$O(1)$	Just peek at heap roots (no removal)

- **Space Complexity: $O(n)$** — we store all n numbers across the two heaps.

Where n is the total number of elements added so far.

5. Alternative Approaches

A) Sorted List with Binary Search Insertion

Maintain a sorted list; use `bisect.insort` to insert in correct position. - `addNum`: **$O(n)$** — finding position is $O(\log n)$, but **shifting** elements is $O(n)$ - `findMedian`: **$O(1)$** — direct index access - Slower for insertions, so the two-heap approach wins.

```
import bisect
class MedianFinder:
    def __init__(self):
        self.arr = []
    def addNum(self, num):
        bisect.insort(self.arr, num) #  $O(n)$  due to shifting
    def findMedian(self):
        n = len(self.arr)
        mid = n // 2
        return self.arr[mid] if n % 2 else (self.arr[mid-1] + self.arr[mid]) / 2
```

B) Self-Balancing BST / Order-Statistic Tree

An AVL/Red-Black tree augmented with subtree sizes can do both operations in $O(\log n)$. Functionally similar to heaps but more complex to implement.

C) Follow-up Optimizations

Follow-up 1 — all numbers in $[0, 100]$: Use **counting / bucket sort** with a fixed array `count[101]`. - `addNum`: **$O(1)$** — just `count[num] += 1` - `findMedian`: **$O(100) = O(1)$** — scan buckets to find the middle.

This is far faster because the value range is tiny and bounded.

Follow-up 2 — 99% of numbers in $[0, 100]$: Use a **hybrid** approach: - A counting array for the common $[0, 100]$ range. - Two separate overflow containers (sorted lists/heaps) for the rare outliers below 0 and above 100. - Combine the bucket counts with the outlier counts to locate the median, keeping the common case $O(1)$.

Codex Solution

1. Intuition

To find the median quickly, we need fast access to the **middle element(s)**.

A sorted array would make `findMedian()` easy, but inserting into the middle of an array costs $O(n)$.

The key idea is to split the numbers into two halves:

lower half	upper half
[small nums]	[large nums]

We maintain:

1. A **max heap** for the lower half
2. Lets us quickly get the largest number in the lower half.
3. In Python, we simulate this using negative values.
4. A **min heap** for the upper half
5. Lets us quickly get the smallest number in the upper half.

Then:

- If the total count is odd, the median is the top of the max heap.
- If the total count is even, the median is the average of both heap tops.

The important invariant is:

```
max_heap contains the smaller half
min_heap contains the larger half

len(max_heap) == len(min_heap)
or
len(max_heap) == len(min_heap) + 1
```

So the max heap is allowed to have one extra element.

2. Approach

Invariants

We keep these rules true after every insertion:

1. Every element in `max_heap` $\leq$ every element in `min_heap`
2. `max_heap` has either the same size as `min_heap` or one extra element

Pseudocode

```
initialize:
    max_heap = []
    min_heap = []

addNum(num):
    if max_heap is empty OR num <= top of max_heap:
        push num into max_heap
    else:
        push num into min_heap

    if max_heap has more than one extra element:
        move top of max_heap to min_heap

    if min_heap has more elements than max_heap:
        move top of min_heap to max_heap

findMedian():
    if max_heap has more elements:
        return top of max_heap

    otherwise:
        return average of top of max_heap and top of min_heap
```

3. Python Solution

```
import heapq

class MedianFinder:
    def __init__(self):
        # max_heap stores the smaller half of numbers.
        # Python only has a min heap, so we store negatives.
        #
        # Example:
        # To store [1, 2, 3] as a max heap,
        # we store [-3, -2, -1].
        # The largest original value is -max_heap[0].
        self.max_heap = []

        # min_heap stores the larger half of numbers normally.
        self.min_heap = []

    def addNum(self, num: int) -> None:
        # Step 1:
        # Decide which heap should receive the new number.
        #
        # If max_heap is empty, or num belongs to the smaller half,
        # put it into max_heap.
        if not self.max_heap or num <= -self.max_heap[0]:
            heapq.heappush(self.max_heap, -num)
        else:
            heapq.heappush(self.min_heap, num)

        # Step 2:
        # Rebalance the heaps if needed.
        #
        # max_heap is allowed to have at most one more element than min_heap.
        if len(self.max_heap) > len(self.min_heap) + 1:
            # Move the largest number from the lower half
            # to the upper half.
            largest_lower = -heapq.heappop(self.max_heap)
            heapq.heappush(self.min_heap, largest_lower)

        # min_heap should never have more elements than max_heap.
        if len(self.min_heap) > len(self.max_heap):
            # Move the smallest number from the upper half
            # to the lower half.
            smallest_upper = heapq.heappop(self.min_heap)
            heapq.heappush(self.max_heap, -smallest_upper)

    def findMedian(self) -> float:
        # If max_heap has one extra element, the total number of elements is odd.
        # The median is the largest element in the lower half.
        if len(self.max_heap) > len(self.min_heap):
            return float(-self.max_heap[0])

        # Otherwise, both heaps have the same size.
        # The median is the average of:
        # - largest number in lower half
        # - smallest number in upper half
        return (-self.max_heap[0] + self.min_heap[0]) / 2.0
```

4. Complexity Analysis

Let n be the number of elements currently stored.

`addNum(num)`

Each insertion uses heap operations:

heappush: $O(\log n)$
heappop: $O(\log n)$

So:

```
Time Complexity:  $O(\log n)$ 
```

```
findMedian()
```

We only look at the top elements of the heaps.

```
Time Complexity:  $O(1)$ 
```

Space Complexity

We store every number in one of the two heaps.

```
Space Complexity:  $O(n)$ 
```

5. Alternative Approaches

1. Sorted List / Array

Keep all numbers sorted.

```
addNum:       $O(n)$       because insertion may require shifting elements
findMedian:  $O(1)$ 
space:        $O(n)$ 
```

This is simple but too slow for many insertions.

2. Balanced Binary Search Tree

Use a balanced BST with order statistics.

```
addNum:       $O(\log n)$ 
findMedian:  $O(\log n)$  or  $O(1)$ , depending on implementation
space:        $O(n)$ 
```

This works well in languages with built-in ordered multisets, but Python does not have one by default.

3. Counting Array for Small Value Range

If all numbers are in range `[0, 100]`, use a frequency array.

```
count = [0] * 101
```

For each number:

```
count[num] += 1
```

To find the median, scan the frequency array until reaching the middle position.

```
addNum:       $O(1)$ 
findMedian:  $O(101)$ , effectively  $O(1)$ 
space:        $O(101)$ , effectively  $O(1)$ 
```

This is excellent when the value range is small.

4. Hybrid Counting for 99% in `[0, 100]`

If 99% of numbers are in `[0, 100]`, we can combine:

- A frequency array for values `0` to `100`
- A structure for numbers below `0`
- A structure for numbers above `100`

For example:

```
below = sorted/heap/BST structure for nums < 0
count[0..100]
above = sorted/heap/BST structure for nums > 100
```

Since most numbers are in `[0, 100]`, median queries often rely mostly on the counting array.

This can be faster in practice when the distribution is highly concentrated.